

Understanding Python Closures

Solutions to Closure Exercises

Exercise 1: Basic Greeting Closure (Scope Warm-up) - Solution

Objective: Build a closure factory `make_greeter(prefix)` that generates customized greeting functions.

Code Solution:

```
"""
Exercise 1: Basic Greeting Closure (Scope Warm-up)
Goal: Create a closure that generates custom greeting functions with a fixed prefix.
"""

def make_greeter(prefix):
    """
    An outer function acting as a closure factory.
    It takes a prefix and returns a function that applies it.
    """
    def greeter(name):
        # prefix is captured from the enclosing scope of make_greeter
        return f"{prefix}, {name}!"
    return greeter

# --- Verification ---
if __name__ == "__main__":
    welcome_greet = make_greeter("Welcome")
    formal_greet = make_greeter("Good morning")

    print(welcome_greet("Alice")) # Expected: Welcome, Alice!
    print(formal_greet("Bob"))    # Expected: Good morning, Bob!
```

Detailed Mechanics:

The `make_greeter` function acts as a factory. When called with `prefix`, it creates and returns a local function `greeter`. The inner function closes over and captures the `prefix` parameter, preserving it in memory so that it remains accessible whenever `welcome_greet` or `formal_greet` is invoked later.

Exercise 2: Stateful Multiplier Factory (Capturing Environment) - Solution

Objective: Implement a multiplier factory `make_multiplier(factor)` that creates functions multiplying their input by a closed-over factor.

Code Solution:

```
"""
Exercise 2: Stateful Multiplier Factory (Capturing Environment)
Goal: Create a function factory that returns a multiplier function.
"""

def make_multiplier(factor):
    """
    Returns a function that multiplies its input by the enclosed 'factor'.
    """
    def multiplier(number):
        # 'factor' is captured from the enclosing environment
        return number * factor
    return multiplier

# --- Verification ---
if __name__ == "__main__":
    double = make_multiplier(2)
    triple = make_multiplier(3)

    print(double(15)) # Expected: 30
    print(triple(15)) # Expected: 45
```

Detailed Mechanics:

The inner function `multiplier` requires access to `factor`. Because `factor` is defined in the parent function's scope, Python packages the active value within the returned function object's environment. This environment keeps the numeric factor alive long after the `make_multiplier` execution frame has closed.

Exercise 3: Simple Event Logger (Tracking State) - Solution

Objective: Build a stateful logger `make_logger()` that preserves an execution history list in memory without using global variables.

Code Solution:

```
"""
Exercise 3: Simple Event Logger (Tracking State)
Goal: Create a logger closure that maintains an execution history list in memory.
"""

def make_logger():
    """
    Returns a function that appends a message to an enclosed list
    and returns the entire list of messages logged so far.
    """
    history = [] # Enclosing scope list

    def log(message):
        # We can append directly because lists are mutable
        history.append(message)
        return list(history) # Return a copy of the list
    return log

# --- Verification ---
if __name__ == "__main__":
    logger = make_logger()

    print(logger("User logged in"))          # Expected: ['User logged in']
    print(logger("Database query run"))      # Expected: ['User logged in', 'Database query run']
    print(logger("User logged out"))        # Expected: ['User logged in', 'Database query run', 'User logged out']
```

Detailed Mechanics:

This highlights a key aspect of Python scopes: lists are mutable objects. Because we are using the `.append()` method, we are modifying the list in-place rather than reassigning the reference. Therefore, the `nonlocal` keyword is not strictly required here, though the list object remains completely hidden and private inside the closure.

Exercise 4: Stateful Counter with nonlocal (Modifying State) - Solution

Objective: Build a function `make_counter()` that implements an isolated stateful counter incremented on each call.

Code Solution:

```
"""
Exercise 4: Stateful Counter with nonlocal (Modifying State)
Goal: Implement a closure that increments and returns a counter.
"""

def make_counter():
    """
    Returns a function that increments and returns an enclosed count.
    Uses the 'nonlocal' keyword to modify an immutable integer.
    """
    count = 0 # Enclosing variable

    def counter():
        nonlocal count # Required to rebind the outer 'count' variable
        count += 1
        return count
    return counter

# --- Verification ---
if __name__ == "__main__":
    counter_a = make_counter()
    counter_b = make_counter()

    print("Counter A:", counter_a()) # Expected: 1
    print("Counter A:", counter_a()) # Expected: 2
    print("Counter B:", counter_b()) # Expected: 1 (isolated state)
    print("Counter A:", counter_a()) # Expected: 3
```

Detailed Mechanics:

Integers are immutable. The operation `count += 1` is equivalent to `count = count + 1`, which is an assignment statement. Without the `nonlocal count` declaration, Python would assume we are trying to create a local variable named `count` before reading it, causing an `UnboundLocalError`. The `nonlocal` keyword forces Python to rebind the outer variable.

Exercise 5: API Rate Limiter (Real-world Utility) - Solution

Objective: Write a rate limiter closure `make_rate_limiter(max_calls)` that restricts function calls, printing a warning and refusing execution once the threshold is crossed.

Code Solution:

```
"""
Exercise 5: API Rate Limiter (Real-world Utility)
Goal: Implement a simple rate limiter closure that restricts function calls.
"""

def make_rate_limiter(max_calls):
    """
    Returns a function wrapper that allows execution up to max_calls.
    Subsequent calls are blocked with a warning message.
    """
    calls_made = 0

    def rate_limiter(func, *args, **kwargs):
        nonlocal calls_made
        if calls_made < max_calls:
            calls_made += 1
            return func(*args, **kwargs)
        else:
            print("WARNING: Rate limit exceeded! Request blocked.")
            return None
    return rate_limiter

# --- Verification ---
if __name__ == "__main__":
    def get_data():
        return "Prisistine API Data"

    limiter = make_rate_limiter(2)

    print(limiter(get_data)) # Expected: Prisistine API Data
    print(limiter(get_data)) # Expected: Prisistine API Data
    print(limiter(get_data)) # Expected: WARNING + None
```

Detailed Mechanics:

This pattern wraps execution. It stores the integer counter `calls_made` in enclosing scope memory, using `nonlocal` to increment it on every successful invocation. Once the counter hits `max_calls`, the gateway blocks all subsequent calls, demonstrating how closures are used as lightweight authorization wrappers around sensitive behavior.

Exercise 6: Memoization Cache (Advanced Closures) - Solution

Objective: Build a caching closure `make_cached(func)` that wraps an expensive function and stores previously computed arguments in a dictionary cache.

Code Solution:

```
"""
Exercise 6: Memoization Cache (Advanced Closures)
Goal: Build a cache decorator-like closure to avoid redundant computations.
"""

def make_cached(func):
    """
    Returns a function that wraps 'func', storing inputs and outputs in an enclosed dictionary.
    """
    cache = {} # Enclosing dictionary for storing results

    def cached_wrapper(x):
        if x not in cache:
            print(f"[CACHE MISS] Calculating result for {x}...")
            cache[x] = func(x)
        else:
            print(f"[CACHE HIT] Retrieving cached result for {x}...")
            return cache[x]
        return cached_wrapper

    # --- Verification ---
    if __name__ == "__main__":
        def expensive_square(x):
            return x * x

        memo_square = make_cached(expensive_square)

        print(memo_square(4)) # Expected: Cache Miss, 16
        print(memo_square(4)) # Expected: Cache Hit, 16
        print(memo_square(5)) # Expected: Cache Miss, 25
```

Detailed Mechanics:

This is the foundational logic of memoization caches. By storing calculated outcomes in the enclosed dictionary, the wrapper eliminates redundant computations. This functional encapsulation is highly popular in caching systems to wrap mathematical or network resources.

Exercise 7: Execution Timer (Prelude to Decorators) - Solution

Objective: Build a closure `make_timer(func)` that intercepts execution to measure and log the timing of a target function in seconds.

Code Solution:

```
"""
Exercise 7: Execution Timer (Prelude to Decorators)
Goal: Build a closure that prints the time taken to execute a target function.
"""
import time

def make_timer(func):
    """
    Returns a wrapped version of 'func' that prints its execution time.
    """
    def timer_wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        end_time = time.perf_counter()
        duration = end_time - start_time
        print(f"[{func.__name__}] Executed in {duration:.6f} seconds.")
        return result
    return timer_wrapper

# --- Verification ---
if __name__ == "__main__":
    def slow_sum(n):
        return sum(range(n))

    timed_sum = make_timer(slow_sum)
    print(timed_sum(1000000)) # Expected: Timing log followed by the sum
```

Detailed Mechanics:

This solution introduces decorators. The `@make_timer` syntax is syntactic sugar for wrapping the function in a closure. By returning `timer_wrapper`, we intercept parameters transparently via positional `*args` and keyword `**kwargs`, measuring times dynamically while preserving function behavior.

Exercise 8: Secure API Token Wrapper (Enclosing Private Properties) - Solution

Objective: Create a client closure `create_api_client(auth_token)` that encloses an authentication token, allowing requests to be sent securely without exposing the raw token attribute.

Code Solution:

```
"""
Exercise 8: Secure API Token Wrapper (Enclosing Private Properties)
Goal: Build an API Client closure where a private token is kept secure and inaccessible from outside
"""

def create_api_client(auth_token):
    """
    Returns an API client with an enclosed authenticating token.
    The returned client function can execute queries but does not leak the token directly.
    """
    # auth_token is safely enclosed and cannot be modified or read from outside.

    def client(endpoint, payload=None):
        print(f"Sending authenticated request to {endpoint}")
        headers = {"Authorization": f"Bearer {auth_token}"}
        # Simulate network request
        return {
            "status": "success",
            "endpoint": endpoint,
            "headers_used": headers,
            "data": "Result of secure endpoint request"
        }
    return client

# --- Verification ---
if __name__ == "__main__":
    client = create_api_client("super_secret_token_123")

    # We can invoke the client behavior
    response = client("/users/me")
    print("Response status:", response["status"])
    print("Authorization header used:", response["headers_used"]["Authorization"])

    # We cannot access the token directly from the client object
    try:
        print(client.auth_token)
    except AttributeError:
        print("Attribute Error raised successfully: 'auth_token' is private to the closure!")
```

Detailed Mechanics:

Unlike OOP Class attributes (which are accessible from outer scopes), closed-over variables like `auth_token` are strictly localized to the closure's private namespace environment. The variable cannot be looked up, modified, or accessed from any code outside of the closure, supplying ultimate safety for variables like security tokens and encryption keys.